LAPORAN PRAKTIKUM PEMROGRAMAN BERORIENTASI OBJEK

Praktikum 9: Unit Testing dengan JUnit 5

Identitas	**Keterangan**
Nama	Ananda Firdaus
NIM	251511003
Kelas	2A
Program Studi	D3 Teknik Informatika
Jurusan	Teknik Komputer dan Informatika
Politeknik	Politeknik Negeri Bandung

Bandung, 11 November 2026

A. Pendahuluan

Praktikum 9 membahas unit testing dengan JUnit 5. Fokus praktikum bukan menambah konsep OOP baru, tetapi membuktikan perilaku class secara otomatis dan berulang. Class yang diuji adalah BankAccount, InsufficientBalanceException, dan SavingsAccount.

Pengujian menggunakan Maven 3.10.0, JDK 26.0.2.1, dan JUnit 5.11.4. Seluruh output pada laporan ini diambil dari eksekusi test yang benar-benar dijalankan. Laporan memuat bukti test pertama, test exception dan integritas state, fixture dan parameterized test, test inheritance dan polymorphism, mini TDD, sintesis hasil, skenario uji, jawaban checkpoint, dan refleksi.

B. Bukti Praktikum

B.1 Test Pertama dan Pola AAA

Test pertama menguji deposit. Object dibuat dengan saldo awal 100.000, lalu deposit 50.000, dan saldo akhir diharapkan 150.000.

```
@Test
void deposit_menambahSaldo() {
    // Arrange
    BankAccount acc =
        new BankAccount("001", "Andi", 100_000);

    // Act
    acc.deposit(50_000);

    // Assert
    assertEquals(150_000, acc.getBalance(), 0.001);
}
```

Kode test deposit_menambahSaldo.

Pola AAA terlihat dari tiga bagian: Arrange menyiapkan object, Act memanggil deposit, dan Assert memeriksa saldo. Hasil eksekusi:

```
Running id.ac.polban.pbo.BankAccountTest
Tests run: 2, Failures: 0, Errors: 0, Skipped: 0
BUILD SUCCESS
```

Hasil test pertama (hijau).

Untuk memastikan test dapat gagal, expected diubah sementara dari 150.000 menjadi 160.000. Hasilnya:

```
expected: <160000.0> but was: <150000.0>
Tests run: 2, Failures: 1, Errors: 0, Skipped: 0
BUILD FAILURE
```

Hasil test saat expected sengaja dibuat salah (merah).

Expected adalah nilai yang diharapkan, sedangkan actual adalah nilai yang benar-benar dihasilkan kode. Setelah expected dikembalikan ke 150.000, kedua test kembali hijau.

B.2 Test Exception dan Integritas State

Method withdraw melempar InsufficientBalanceException ketika saldo tidak cukup. Exception ini adalah checked exception karena extends Exception.

```
public class InsufficientBalanceException extends Exception {
    public InsufficientBalanceException(String message) {
        super(message);
    }
}

public void withdraw(double amount)
        throws InsufficientBalanceException {
    if (amount <= 0)
        throw new IllegalArgumentException(
            "Jumlah penarikan harus lebih dari 0");
    if (amount > balance)
        throw new InsufficientBalanceException(
            "Saldo tidak mencukupi");
    balance -= amount;
}
```

Test berikut memastikan exception muncul dan saldo tidak berubah.

```
@Test
void withdraw_melebihiSaldo_ditolak() {
    BankAccount acc =
        new BankAccount("001", "Andi", 100_000);

    assertThrows(
        InsufficientBalanceException.class,
        () -> acc.withdraw(200_000)
    );

    assertEquals(100_000, acc.getBalance(), 0.001);
}
```

Kode test withdraw melebihi saldo.

Hasil eksekusi:

```
Tests run: 5, Failures: 0, Errors: 0, Skipped: 0
BUILD SUCCESS
```

Hasil test exception dan state (hijau).

Untuk membuktikan pentingnya pemeriksaan saldo, urutan withdraw sengaja dibalik: saldo dikurangi lebih dulu, baru dicek. Hasilnya:

```
withdraw_melebihiSaldo_ditolak:35
expected: <100000.0> but was: <-100000.0>
Tests run: 5, Failures: 1, Errors: 0, Skipped: 0
BUILD FAILURE
```

Kegagalan test saat saldo dikurangi sebelum validasi.

Exception tetap muncul, tetapi saldo sudah rusak menjadi -100.000. Assert setelah assertThrows menangkap bug ini. Setelah urutan dikembalikan, seluruh test hijau lagi.

B.3 Fixture dan Parameterized Test

Setup yang berulang dipindahkan ke @BeforeEach. JUnit menjalankan setUp sebelum setiap test, sehingga setiap test mendapat object baru. Test deposit dan withdraw yang sebelumnya membuat object sendiri sekarang memakai sender dari fixture.

```
private BankAccount sender;
private BankAccount receiver;

@BeforeEach
void setUp() {
    sender = new BankAccount("001", "Andi", 100_000);
    receiver = new BankAccount("002", "Budi", 0);
}
```

Setup fixture dengan @BeforeEach.

Parameterized test digunakan untuk data invalid, nilai boundary, dan beberapa kombinasi withdraw.

```
@ParameterizedTest(name = "setoran {0} ditolak")
@ValueSource(doubles = {0, -1, -50_000})
void deposit_tidakValid(double amount) {
    assertThrows(
        IllegalArgumentException.class,
        () -> sender.deposit(amount)
    );

    assertEquals(100_000, sender.getBalance(), 0.001);
}

@ParameterizedTest(name = "setoran {0} diterima")
@ValueSource(doubles = {0.001, 1})
void deposit_batasTerkecil_diterima(double amount) {
    assertDoesNotThrow(() -> sender.deposit(amount));
    assertEquals(100_000 + amount, sender.getBalance(), 0.001);
}

@ParameterizedTest(name = "saldo {0} - tarik {1} = {2}")
@CsvSource({
    "100000, 10000, 90000",
    "100000, 100000, 0",
    "50000, 25000.5, 24999.5"
})
void withdraw_berbagaiNilai(
        double awal, double tarik, double sisa)
        throws InsufficientBalanceException {
    BankAccount acc =
        new BankAccount("009", "Dewi", awal);

    acc.withdraw(tarik);

    assertEquals(sisa, acc.getBalance(), 0.001);
}
```

Parameterized test dengan @ValueSource dan @CsvSource.

Nilai boundary 0.001 ditambahkan pada deposit karena nilai itu adalah batas terkecil yang masih diterima. Untuk withdraw, kombinasi 100.000 - 100.000 = 0 menguji tarik tepat seluruh saldo. Hasil eksekusi:

```
Tests run: 13, Failures: 0, Errors: 0, Skipped: 0
BUILD SUCCESS
```

Hasil setelah @BeforeEach dan parameterized test.

B.4 Inheritance dan Polymorphism

SavingsAccount mewarisi BankAccount dan meng-override withdraw dengan aturan saldo minimum 50.000.

```
public class SavingsAccount extends BankAccount {
    public static final double MIN_BALANCE = 50_000;
    private final double interestRate;

    public SavingsAccount(String accountNumber, String owner,
                          double initialBalance,
                          double interestRate) {
        super(accountNumber, owner, initialBalance);
        if (interestRate < 0)
            throw new IllegalArgumentException(
                "Bunga tidak boleh negatif");
        this.interestRate = interestRate;
    }

    @Override
    public void withdraw(double amount)
            throws InsufficientBalanceException {
        if (getBalance() - amount < MIN_BALANCE)
            throw new InsufficientBalanceException(
                "Saldo minimum Rp50.000 harus dipertahankan");
        super.withdraw(amount);
    }

    public void addInterest() {
        balance += balance * interestRate;
    }
}
```

Implementasi SavingsAccount.

Test berikut memakai reference BankAccount, tetapi object yang dibuat adalah SavingsAccount.

```
@Test
void aturanSubclass_berlakuLewatReferenceInduk() {
    BankAccount acc = new SavingsAccount(
        "003", "Citra", 100_000, 0.05);

    assertThrows(
        InsufficientBalanceException.class,
        () -> acc.withdraw(60_000)
    );

    assertEquals(100_000, acc.getBalance(), 0.001);
}
```

Test aturan subclass melalui reference induk.

Saat acc.withdraw(60_000) dipanggil, compiler memeriksa method withdraw pada BankAccount. Pada runtime, object sebenarnya adalah SavingsAccount, sehingga override SavingsAccount yang dijalankan. Ini adalah dynamic dispatch. Hasil eksekusi:

```
Running id.ac.polban.pbo.BankAccountTest
Tests run: 13, Failures: 0, Errors: 0, Skipped: 0
Running id.ac.polban.pbo.SavingsAccountTest
Tests run: 4, Failures: 0, Errors: 0, Skipped: 0
Tests run: 17, Failures: 0, Errors: 0, Skipped: 0
BUILD SUCCESS
```

Hasil test inheritance dan polymorphism.

Test lain memeriksa withdraw tepat di batas minimum 50.000, perhitungan bunga, dan penolakan bunga negatif. Semua test tersebut juga hijau.

B.5 Mini TDD: Transfer dengan Target Null

Baseline transfer menarik saldo pengirim lalu menyetor ke target, tanpa validasi target.

```
public void transfer(BankAccount target, double amount)
        throws InsufficientBalanceException {
    withdraw(amount);
    target.deposit(amount);
}
```

Test untuk aturan baru ditulis lebih dulu.

```
@Test
void transfer_targetNull_ditolak() {
    assertThrows(
        IllegalArgumentException.class,
        () -> sender.transfer(null, 20_000)
    );

    assertEquals(100_000, sender.getBalance(), 0.001);
}
```

Test transfer dengan target null.

Fase RED: test gagal karena kode melempar NullPointerException, bukan IllegalArgumentException.

```
Tests run: 19, Failures: 1, Errors: 0, Skipped: 0
expected: <java.lang.IllegalArgumentException>
but was: <java.lang.NullPointerException>
BUILD FAILURE
```

Fase RED mini TDD.

Pemeriksaan saldo melalui jshell menunjukkan saldo pengirim sudah berkurang sebelum exception muncul.

```
Exception: NullPointerException
Saldo setelah gagal: 80000.0
```

Bukti saldo pengirim berkurang saat transfer gagal.

Fase GREEN: validasi target ditambahkan sebelum withdraw.

```
public void transfer(BankAccount target, double amount)
        throws InsufficientBalanceException {
    if (target == null)
        throw new IllegalArgumentException(
            "Rekening tujuan wajib diisi");
    withdraw(amount);
    target.deposit(amount);
}
```

```
Tests run: 19, Failures: 0, Errors: 0, Skipped: 0
BUILD SUCCESS
```

Fase GREEN mini TDD.

Setelah GREEN, tidak ada duplikasi yang perlu dirapikan. Siklus RED-GREEN-REFACTOR selesai.

B.6 Verifikasi Akhir

Verifikasi akhir dijalankan dengan mvn test setelah seluruh test ditambahkan.

```
Running id.ac.polban.pbo.BankAccountTest
Tests run: 15, Failures: 0, Errors: 0, Skipped: 0
Running id.ac.polban.pbo.SavingsAccountTest
Tests run: 4, Failures: 0, Errors: 0, Skipped: 0
Tests run: 19, Failures: 0, Errors: 0, Skipped: 0
BUILD SUCCESS
```

Hasil verifikasi akhir, 19 test hijau.

C. Sintesis Hasil Praktikum

Kasus	**Behavior yang dibuktikan**	**Assertion / mekanisme**
Deposit	Saldo bertambah sesuai amount	assertEquals
Withdraw invalid	Exception muncul dan saldo tetap	assertThrows + assertEquals
Parameterized	Satu behavior diuji dengan beberapa data	@ValueSource / @CsvSource
SavingsAccount	Override tetap berlaku via reference BankAccount	assertThrows / assertEquals
Mini TDD	Test gagal sebelum perbaikan dan lulus sesudahnya	Red -> Green -> Refactor

Seluruh kasus di atas dijalankan dan menghasilkan 19 test hijau.

D. Skenario Uji

ID	Skenario	Expected Result	Hasil
T9-01	deposit 50.000 pada saldo 100.000	saldo 150.000	Lulus
T9-02	expected sengaja salah	test merah dengan expected/actual	Terbukti merah
T9-03	withdraw melebihi saldo	exception; saldo tidak berubah	Lulus
T9-04	withdraw valid	tidak melempar; saldo berkurang	Lulus
T9-05	deposit 0/negatif	IllegalArgumentException	Lulus
T9-06	SavingsAccount via BankAccount	override subclass tetap berlaku	Lulus
T9-07	transfer target null sebelum perbaikan	RED	Terbukti RED
T9-08	transfer target null sesudah perbaikan	GREEN; saldo tetap	Lulus

E. Jawaban Checkpoint

Checkpoint 1

Jika program hanya dijalankan sekali melalui main(), yang terbukti hanya satu jalur sukses. Kondisi yang belum terbukti antara lain constructor menolak nomor rekening kosong, constructor menolak saldo negatif, deposit menolak 0 atau negatif, getter mengembalikan nilai yang benar, dan perilaku tetap sama saat dijalankan ulang atau setelah kode diubah.

Checkpoint 2

Expected adalah nilai yang diharapkan, sedangkan actual adalah nilai yang benar-benar dihasilkan kode. Dengan sengaja membuat expected salah, kita memastikan test benar-benar bisa gagal dan melihat pesan diagnostiknya. Test yang selalu hijau belum tentu menguji sesuatu.

Checkpoint 3

Saldo perlu diperiksa karena exception hanya membuktikan operasi ditolak, bukan membuktikan object tetap utuh. Jika withdraw mengurangi saldo sebelum validasi, exception tetap muncul tetapi saldo sudah rusak. Assert saldo setelah assertThrows menangkap bug tersebut.

Checkpoint 4

@BeforeEach membuat object baru sebelum setiap test sehingga test tidak berbagi state dan tidak bergantung pada urutan. Tidak semua Arrange harus dipindahkan ke sana karena sebagian test membutuhkan data khusus, seperti saldo 50.000, saldo -1, atau data CSV yang berbeda.

Checkpoint 5

Parameterized test tepat ketika perilakunya sama dan hanya data yang berubah. Test terpisah lebih mudah dibaca ketika perilaku atau setup berbeda, atau ketika setiap kasus membutuhkan nama dan penjelasan sendiri.

Checkpoint 6

Konsep yang terlihat adalah polymorphism, khususnya dynamic dispatch. Tipe reference BankAccount menentukan method yang boleh dipanggil saat compile, sedangkan tipe object sebenarnya, yaitu SavingsAccount, menentukan override yang dijalankan saat runtime.

Checkpoint 7

Melihat RED lebih dulu membuktikan test benar-benar menguji perilaku yang belum ada. Saat test yang sama berubah GREEN setelah perbaikan minimal, kita tahu perbaikan itu yang membuat perilaku lulus, bukan kebetulan. Ini juga mengurangi risiko test yang selalu hijau tanpa menguji apa pun.

F. Refleksi

1. Apa perbedaan antara kode yang berhasil dijalankan dan behavior yang teruji?

Kode yang berhasil dijalankan hanya menunjukkan satu output pada satu kondisi. Behavior yang teruji berarti perilaku tersebut sudah diperiksa otomatis dengan expected yang jelas, sehingga bisa diulang dan dicek kembali saat kode berubah.

2. Mengapa unit test sebaiknya menguji behavior publik object, bukan field private?

Unit test sebaiknya menguji perilaku publik karena itu adalah kontrak yang dipakai pengguna class. Field private adalah detail implementasi yang bisa berubah tanpa mengubah perilaku. Menguji field private membuat test rapuh dan menghambat refactoring.

3. Pada withdraw gagal, apa hubungan exception dengan encapsulation dan integritas state?

Exception menunjukkan operasi gagal. Encapsulation menjaga state tetap valid. Jika withdraw gagal tetapi saldo sudah berkurang, integritas state rusak. Karena itu test harus memeriksa saldo setelah exception.

4. Mengapa satu object bersama untuk seluruh test dapat membuat hasil test tidak independen?

Satu object bersama membuat perubahan dari satu test terbawa ke test lain. Hasil test jadi bergantung pada urutan dan bisa lulus atau gagal secara tidak konsisten. @BeforeEach membuat object baru agar setiap test independen.

5. Kapan parameterized test membantu, dan kapan test terpisah lebih mudah dibaca?

Parameterized test membantu saat banyak data menguji behavior yang sama, sehingga duplikasi kode berkurang. Test terpisah lebih baik saat behavior atau setup berbeda, atau saat data terlalu kompleks sehingga tabel sulit dibaca.

6. Apa yang dibuktikan ketika variabel BankAccount menjalankan aturan withdraw SavingsAccount?

Yang dibuktikan adalah polymorphism melalui dynamic dispatch. Meskipun reference bertipe BankAccount, object SavingsAccount tetap menjalankan override withdraw miliknya. Tipe object sebenarnya menentukan method yang dieksekusi.

7. Apa manfaat melihat test gagal pada fase RED sebelum menulis implementasi?

Melihat RED memberi bukti bahwa test benar-benar menguji sesuatu yang belum ada. Saat test berubah GREEN setelah perbaikan, kita tahu perbaikan tersebut menyelesaikan masalah. Tanpa RED, test bisa saja selalu hijau karena tidak menguji apa pun.

8. Lengkapi dengan bahasa Anda sendiri: "Unit test bukan untuk membuktikan program tidak pernah salah, tetapi ..."

Unit test bukan untuk membuktikan program tidak pernah salah, tetapi untuk mendeteksi kesalahan lebih awal, menjaga perilaku yang sudah ditetapkan, dan memberi rasa aman saat kode diubah atau di-refactor.